ReSECT: A Real-to-Sim Cohesive Zone Framework for Differentiable Topology-Changing Cutting in Robotic Surgery

Yunxin Fan, Doug L. James, Monroe Kennedy III, Nikolas Blevins

***Abstract*— Deformable object simulation, especially cutting and debulking, remains a challenging task due to its natural property of topology changing. Furthermore, this simulation mostly in forward manner where the backpropagation is not allowed, which is not applicable for leanring based task integration. Surgical scene, tissue or tumor property is always ill-define/evaluate domain, where the mechanics of tissue and tumor is always hard to calibrate. In this paper, we proposed a**

I. Related Work

1. Differentiable continuum simulation 2. Cutting in PBD / continuum simulation

II. Methodology

A. Differentiable Shape-Matching PBD Approach

For completeness, we briefly review the shape-matching PBD approach used in our soft-tissue simulator. We adopt this solver because its per-vertex participation weights directly modulate the deformation dynamics, providing the interface through which the continuous cutting field in §II-B is applied.

The simulated soft tissue is represented by a tetrahedral mesh $\mathcal{M} = (\mathcal{V}, \mathcal{E}, \mathcal{F}, \mathcal{T})$, with one simulation particle located at each vertex $i \in \mathcal{V}$. The vertices are grouped into overlapping clusters $\{C_1, C_2, \ldots\}$, each consisting of a small set of nearby vertices with a remembered rest shape. For each $i \in C$, cluster C stores a current position $\mathbf{p}_i \in \mathbb{R}^3$, a rest position $\bar{\mathbf{p}}_i \in \mathbb{R}^3$, and a participation weight $w_i \in \mathbb{R}_{\geq 0}$. $\bar{\mathbf{p}}_i \in \mathbb{R}^3$ is fixed at mesh initialization, while $\mathbf{p}_i$ and w_i evolve over time. At each substep, the cluster solve recovers the best-fit rigid rotation $\mathbf{R}_C \in \mathrm{SO}(3)$ that aligns the deformed cluster with its rest shape in a weighted least-squares sense.

(i) We first compute the weighted centroids of the current and rest configurations, $\boldsymbol{\mu}_C, \bar{\boldsymbol{\mu}}_C \in \mathbb{R}^3$,

$$\boldsymbol{\mu}_C = \frac{\sum_{i \in C} w_i \mathbf{p}_i}{\sum_{i \in C} w_i}, \qquad \bar{\boldsymbol{\mu}}_C = \frac{\sum_{i \in C} w_i \bar{\mathbf{p}}_i}{\sum_{i \in C} w_i}. \tag{1}$$

These centroids remove the translational component of the cluster alignment.

(ii) We then form the cross-covariance matrix $\mathbf{A}_C \in \mathbb{R}^{3\times 3}$,

$$\mathbf{A}_C = \sum_{i \in C} w_i \, (\mathbf{p}_i - \boldsymbol{\mu}_C)(\bar{\mathbf{p}}_i - \bar{\boldsymbol{\mu}}_C)^\top, \tag{2}$$

which relates the current cluster deviations to their corresponding rest-shape deviations.

(iii) The best-fit rigid rotation is obtained by applying polar decomposition to the cross-covariance matrix, $\mathbf{A}_C = \mathbf{R}_C \mathbf{S}_C$. The rotation $\mathbf{R}_C \in \mathrm{SO}(3)$ is retained, while the symmetric stretch $\mathbf{S}_C$ is discarded because shape matching uses only the rigid component of the cluster motion. The translation follows from centroid alignment, $\mathbf{t}_C = \boldsymbol{\mu}_C - \mathbf{R}_C \bar{\boldsymbol{\mu}}_C$.

(iv) This rigid alignment defines a goal position and a shape-matching correction for each vertex $i \in C$:

$$\mathbf{g}_{i,C} = \mathbf{R}_C \bar{\mathbf{p}}_i + \mathbf{t}_C, \qquad \Delta_{i,C} = w_i(\mathbf{g}_{i,C} - \mathbf{p}_i) \in \mathbb{R}^3. \tag{3}$$

The key property that makes shape matching useful for our formulation is its explicit participation weight w_i. As shown in (1)–(3), the same scalar weight appears throughout the cluster solve: it determines how strongly vertex i contributes to the current and rest centroids, how much it influences the fitted rotation through the covariance matrix $\mathbf{A}_C$, and how strongly it is pulled toward the resulting goal position. Thus, changing w_i does not only scale a final displacement; it continuously changes the vertex's role in the entire shape-matching computation. This gives a natural interface for differentiable cutting: instead of making a discrete present-or-removed decision, we assign each vertex a continuous participation value and let it modulate the cluster solve itself. Intact vertices participate with $w_i \approx 1$, removed vertices with $w_i \approx 0$, and intermediate values represent partial participation near the cut boundary. The resulting map from $\{w_i\}$ to the cluster corrections is differentiable away from standard degeneracies, such as collinear vertices, repeated singular values of $\mathbf{A}_C$, or vanishing total weight. We handle these cases with two small regularizations: the centroid denominator is floored by $\max(\sum_{i \in C} w_i, \varepsilon)$, and $\varepsilon \mathbf{I}_3$ (with $\varepsilon = 10^{-6}$) is added to $\mathbf{A}_C$ before the polar iteration.

At each substep, we perform M Jacobi iterations of the cluster solve in parallel on the GPU: every cluster reads a shared state, and the overlapping per-vertex corrections are gathered into a fresh state for the next iteration. At Jacobi iteration m, let $\mathbf{p}^m$ denote the current vertex positions. After all cluster corrections are computed, the update for vertex i is

$$\mathbf{p}_i^{m+1} = \mathbf{p}_i^m + \frac{k}{|C(i)|} \sum_{C \in C(i)} \Delta_{i,C}, \tag{4}$$

where $m = 0, \ldots, M-1$, $C(i)$ is the set of clusters containing vertex i, and each $\Delta_{i,C}$ is evaluated at the current iterate $\mathbf{p}^m$.

B. Continuous Topology Change via Soft Cluster Weights

The previous section shows that the shape-matching solver can be modulated by a continuous per-vertex participation weight w_i. We now define how this weight is generated from the cutting geometry. Rather than treating cutting as a discrete topological event, we assign each edge a continuous survival score and propagate the resulting damage to the vertex weights

used by the cluster solve. Let F denote the event that a topological feature remains intact under an intruding body I. We define the survival score first on edges, $P(F \mid e, I)$, because edges are the primitive units from which face- and tetrahedron-level survival can be composed. Specifically, a face remains intact only if all of its boundary edges remain intact, and a tetrahedron remains intact only if all six of its edges remain intact. Assuming these edge events are conditionally independent given I, we use the modeling approximation

$$\begin{aligned} P(F|f, I) &= \prod_{e \in f} P(F|e, I), \\ P(F|T, I) &= \prod_{e \in T} P(F|e, I). \end{aligned} \tag{5}$$

so faces f and tetrahedra T do not require separate cut variables; their survival scores are induced by the edge scores.

We define the edge survival score from the interaction between an edge and an intruder. Let $I = (\mathbf{c}_I, \rho_I)$ denote a spherical intruder with center $\mathbf{c}_I$ and radius $\rho_I \geq 0$, and let $B(\mathbf{c}, r)$ denote the closed ball with center $\mathbf{c}$ and radius r. During cutting, the intruder corresponds to the tool geometry. For an edge $e = (a, b)$ with endpoints $a, b \in \mathcal{V}$, we define its midpoint and half-length as

$$\mathbf{p}_e = \frac{1}{2}(\mathbf{p}_a + \mathbf{p}_b), \qquad r_e = \frac{1}{2}\|\mathbf{p}_a - \mathbf{p}_b\|.$$

We approximate the edge by its diametral sphere $B(\mathbf{m}_e, r_e)$ and measure its signed separation from the intruder by

$$d(e, I) = \|\mathbf{p}_e - \mathbf{c}_I\| - \rho_I - r_e, \tag{6}$$

which is positive when the two spheres are disjoint and negative when they overlap.

The edge survival score is then defined as

$$P_e := P(F \mid e, I) = \sigma\left(\kappa \, \frac{d(e, I)}{r_e}\right), \qquad P_e \in (0, 1), \tag{7}$$

where $\sigma(x) = 1/(1 + \exp(-x))$ and $\kappa > 0$ controls the sharpness of the transition. The normalized distance $d(e, I)/r_e$ makes the score invariant to uniform mesh scaling. Thus, P_e approaches 1 far from the tool, approaches 0 under deep penetration, and equals $\frac{1}{2}$ at tangency.

For numerical smoothness, we replace the nonsmooth norm and edge length near zero by the smoothed form $x \mapsto \sqrt{x^2 + \varepsilon_{\text{geo}}^2}$, with $\varepsilon_{\text{geo}} = 10^{-4}$. We also compute a rest baseline P_e^{rest} by evaluating (7) once at initialization using the nearest non-adjacent vertex as a point intruder. This baseline normalizes the subsequent damage update by the local mesh spacing. The construction is similar in spirit to the minimum-ball score used in DMesh++ [**?**], but is applied at the edge level within the local 1-ring.

A cut is then the cumulative effect of $P(F)$ on participation, propagated through a three-link chain $P(F)_e \to \delta_e \to \delta_i \to \alpha_i$. First, the score drives a per-edge damage $\delta_e \geq 0$ that can only grow, encoding the irreversibility of cutting:

$$\delta_e^{\text{next}} = \delta_e^{\text{prev}} + \sigma_+\left(\hat{\delta}_e - \delta_e^{\text{prev}}\right), \hat{\delta}_e := \sigma_+\left(1 - \frac{P(F)_e}{P(F)_e^{\text{rest}}}\right), \tag{8}$$

where the superscripts prev/next index successive substeps, $\hat{\delta}_e \geq 0$ is the instantaneous damage target, and the softplus $\sigma_+(x) = \frac{1}{k_s}\log(1 + \exp(k_s x))$, $\sigma_+ : \mathbb{R} \to \mathbb{R}_{>0}$, smoothly approximates $\max(x, 0)$ with sharpness $k_s > 0$. The outer softplus is a differentiable stand-in for the positive part $[\,\hat{\delta}_e - \delta_e^{\text{prev}}]_+$, with $[x]_+ := \max(x, 0)$, so δ_e tracks the running maximum of its target $\hat{\delta}_e$ while staying differentiable. Smoothness carries a price: the softplus has a small positive tail $\sigma_+(0) = \log 2/k_s$, so δ_e drifts upward even where $\hat{\delta}_e < \delta_e^{\text{prev}}$, at a rate bounded by $\log 2/k_s$ per substep—negligible over our cutting horizons (§**??**). Second, edge damage averages onto vertices under the same smooth clamp,

$$\delta_i^{\text{next}} = \delta_i^{\text{prev}} + \sigma_+\left(\bar{\delta}_i - \delta_i^{\text{prev}}\right), \bar{\delta}_i := \frac{1}{|E(i)|} \sum_{e \in E(i)} \delta_e^{\text{next}}, \tag{9}$$

where $E(i) = \{e \in \mathcal{E} : i \in e\}$ is the set of edges incident to vertex i and $\bar{\delta}_i$ their mean damage; averaging (rather than summing) keeps the threshold τ below comparable for interior and boundary vertices. Third, the vertex damage becomes the participation mask through a sigmoid,

$$\alpha_i = \sigma(-s\,(\delta_i - \tau)) \in [0, 1], \tag{10}$$

with sharpness $s > 0$ and threshold τ: $\alpha_i = 1$ is a fully participating vertex, $\alpha_i = 0$ a removed one, and intermediate values a smooth interpolation between them.

The chain above produces a per-vertex mask α_i; the solver of §II-A consumes a per-vertex weight w_i. We set them equal:

$$w_i := \alpha_i. \tag{11}$$

This single assignment is the entire coupling between topology and dynamics. Because w_i already runs through the centroid, the covariance, and the correction, a cut vertex ($\alpha_i = 0$) leaves all of its clusters at once, a partially cut vertex stays partially present, and an intact one ($\alpha_i = 1$) recovers the standard shape-matching solve—with no branch, no auxiliary threshold, and no remeshing. The discrete act of cutting has become one differentiable assignment.

a) Scope.

The mesh and cluster graph are fixed for the entire simulation; cutting is expressed purely as a loss of dynamic participation, never as remeshing or fragment separation. This is a deliberate design choice—a fixed computation graph is what makes reverse-mode differentiation possible, and a participation field is sufficient for the quasi-static cavity geometry our system identification targets. One consequence is worth stating plainly: within a single cluster straddling the cut, two still-intact subregions remain bound by the same $\mathbf{R}_C$ and $\boldsymbol{\mu}_C$ and cannot separate kinematically. We therefore use compact clusters (BFS-2 neighborhoods, ∼30 vertices, ∼1.5 mm mean edge length), so the low-participation band a tool leaves behind is thin relative to typical cut depths ($\geq$ 5 mm): coupling is attenuated within each straddled cluster and effectively severed between clusters, with empirical cavity-formation behaviour reported in §**??**. Genuine fragment detachment—independent rigid-body motion for severed pieces—lies outside this formulation and outside the quasi-static regime it is built for.

Algorithm 1 gives one full substep of the integrated solver—advancing the state $(\mathbf{p}, \mathbf{v}, \alpha, \delta_e, \delta_i)$ from substep prev to next over a step dt, with $\mathbf{v}_i$ the per-vertex velocity; Figure **??** illustrates it.

Algorithm 1: Differentiable PBD substep with $P(F)$ cutting and cycle-scoped buffers.

Input : $\mathbf{p}^{\text{prev}}, \mathbf{v}^{\text{prev}}, \alpha^{\text{prev}}, \delta^{\text{e,prev}}, \delta^{\text{v,prev}}, (\mathbf{c}_I, \rho_I)$, dt, M; parameters $\theta = (\kappa, s, \tau, k)$; softplus sharpness k_s fixed
Output : $\mathbf{p}^{\text{next}}, \mathbf{v}^{\text{next}}, \alpha^{\text{next}}, \delta^{\text{e,next}}, \delta^{\text{v,next}}$

$\mathbf{p}^{(0)} \leftarrow \mathbf{p}^{\text{prev}} + dt \cdot \mathbf{v}^{\text{prev}}$ // prediction step
for $e \in \mathcal{E}$ **parallel do**
  $P(F)_e \leftarrow \sigma(\kappa\, d(e, I)/r_e)$ // Eq. 7
  $\hat{\delta}_e \leftarrow \sigma_+\big(1 - P(F)_e / P(F)_e^{\text{rest}}\big)$ // target damage
  $\delta_e^{\text{e,next}} \leftarrow \delta_e^{\text{e,prev}} + \sigma_+\big(\hat{\delta}_e - \delta_e^{\text{e,prev}}\big)$ // Eq. 8, fresh buffer
for $i \in \mathcal{V}$ **parallel do**
  $\bar{\delta}_i \leftarrow \frac{1}{|E(i)|} \sum_{e \in E(i)} \delta_e^{\text{e,next}}$
  $\delta_i^{\text{v,next}} \leftarrow \delta_i^{\text{v,prev}} + \sigma_+\big(\bar{\delta}_i - \delta_i^{\text{v,prev}}\big)$ // Eq. 9
  $\alpha_i^{\text{next}} \leftarrow \sigma\Big(-s\,(\delta_i^{\text{v,next}} - \tau)\Big)$ // Eq. 10
for $c \leftarrow 0$ **to** $K-1$ **do**
  for *cluster* C **parallel do**
    $w_i \leftarrow \alpha_i^{\text{next}}$ // bridge, Eq. 11
    $\boldsymbol{\mu}_C \leftarrow \big(\sum_{i \in C} w_i\big)^{-1} \sum_{i \in C} w_i\, \mathbf{p}_i^{(c)}$
    $\mathbf{A}_C \leftarrow \sum_{i \in C} w_i\, (\mathbf{p}_i^{(c)} - \boldsymbol{\mu}_C)(\bar{\mathbf{p}}_i - \bar{\boldsymbol{\mu}}_C)^{\top} + \varepsilon\, \mathbf{I}_3$
    $\mathbf{R}_C \leftarrow \text{polar}(\mathbf{A}_C)$
    $\Delta_i^{(C)} \leftarrow w_i \Big(\mathbf{R}_C(\bar{\mathbf{p}}_i - \bar{\boldsymbol{\mu}}_C) + \boldsymbol{\mu}_C - \mathbf{p}_i^{(c)}\Big)$ // Eq. 3
  $\mathbf{p}_i^{(c+1)} \leftarrow \mathbf{p}_i^{(c)} + \frac{k}{|C(i)|} \sum_{C \in \mathcal{C}(i)} \Delta_i^{(C)}$ // Eq. 4; fresh buffer
$\mathbf{p}^{\text{next}} \leftarrow \mathbf{p}^{(K)}, \quad \mathbf{v}^{\text{next}} \leftarrow (\mathbf{p}^{\text{next}} - \mathbf{p}^{\text{prev}})/dt$
return $\mathbf{p}^{\text{next}}, \mathbf{v}^{\text{next}}, \alpha^{\text{next}}, \delta^{\text{e,next}}, \delta^{\text{v,next}}$

One implementation detail makes Algorithm 1 differentiable: every state buffer is written to a fresh allocation rather than modified in place. With K Jacobi iterations this means $K+1$ position buffers—iteration c reads $\mathbf{p}^{(c)}$ and writes $\mathbf{p}^{(c+1)}$—and the same pattern for $\alpha, \delta_e, \delta_i$, and the per-iteration cluster rotations. In-place writes to tensors whose pre-update values the backward pass still needs would either raise tape errors or silently zero out gradients [**?**]; with correct buffering the gradient $\partial \mathcal{L}/\partial \theta$ is automatic, and §**??** confirms it matches finite differences to machine precision.

C. Joint Per-Particle Parameter Calibration

The solver of Algorithm 1 is a differentiable forward map from parameters to observed deformation. We *calibrate* these parameters from observation rather than identifying physical material constants: the learned values represent per-particle settings that, together with our forward simulator, reproduce the observed trajectory. This framing is consistent with Liang [**?**], whose constraint weights k are similarly PBD parameters rather than material moduli.

a) Parameter partition.

We partition parameters into per-particle calibration variables and frozen global hyperparameters:

$$\underbrace{(\mathbf{k}_{pp}, \boldsymbol{\tau}_{pp})}_{\text{calibrated}} \in \mathbb{R}^N \times \mathbb{R}^N, \qquad \underbrace{(\alpha_{\min},\ s,\ \tau_{\text{global}})}_{\text{frozen}} \in \mathbb{R}^3. \quad (12)$$

Here $\mathbf{k}_{pp}$ is the per-particle shape-matching weight (a PBD relaxation rate; see §**??** for physical interpretation) and $\boldsymbol{\tau}_{pp}$ is the per-particle aspiration threshold of the soft-removal sigmoid (10). The geometric P(F) sharpness $\alpha_{\min}$ and the alive-sigmoid sharpness s are calibrated once globally and held fixed during per-particle recovery, as their joint identifiability with the spatial fields is weak under our cube-and-knife setup. The global threshold τ_{global} serves only as a numeric fallback when the per-particle path is disabled.

b) Sigmoid-bounded parameterization.

Direct optimization of $\mathbf{k}_{pp}, \boldsymbol{\tau}_{pp}$ as unconstrained tensors empirically diverges at the steeper sigmoid operating points required for crisp removal ($s \approx 100$). Following Liang [**?**], we reparameterize via sigmoid wrap. For each field $\mathbf{x}_{pp} \in \{\mathbf{k}_{pp}, \boldsymbol{\tau}_{pp}\}$ with prescribed bounds $[l_x, u_x]$, we optimize an auxiliary unconstrained variable $\mathbf{z}_x \in \mathbb{R}^N$ and recover $\mathbf{x}_{pp}$ at every forward call:

$$\mathbf{x}_{pp,i} = l_x + (u_x - l_x)\, \sigma(z_{x,i}). \quad (13)$$

This bounds $\mathbf{x}_{pp}$ smoothly without clipping, providing saturation-induced soft gradient clipping near the bounds, and admits significantly larger Adam step sizes than the raw-tensor parameterization.

c) Outer optimization loop.

Given an observed sequence $\{\mathbf{z}_t\}_{t=1}^{T}$ (per-particle positions and alive scalars from a synthetic ground-truth simulation, or in future work, point-cloud and force observations from a phantom), we minimize

$$\mathcal{L}_{\text{total}} \;=\; \mathcal{L}_{\text{gap}} + \lambda_{\text{smooth}}\, \mathcal{L}_{\text{smooth}}, \quad (14)$$

where $\mathcal{L}_{\text{gap}} = \|\Delta_{\mathbf{x}}^*\|^2 + \|\Delta_{\mathbf{d}}^*\|^2$ is the inner residual mapping norm (15; see paragraph below) and $\mathcal{L}_{\text{smooth}} = \sum_{(i,j) \in \mathcal{E}} |x_i - x_j|$ is the edge-graph spatial smoothness regularizer of Liang [**?**] Eq. 9. Adam updates the two auxiliary fields $(\mathbf{z}_k, \mathbf{z}_\tau)$ with independent learning rates, reflecting the different gradient scales of the stiffness and threshold paths through the forward sim.

d) Inner residual mapping (Liang-aligned).

At each frame t we solve a small geometric residual mapping inspired by Liang [**?**] Eq. 6:

$$(\Delta_{\mathbf{x}}^*, \Delta_{\mathbf{d}}^*) \;=\; \arg\min_{\Delta_{\mathbf{x}}, \Delta_{\mathbf{d}}} \; \mathcal{L}_{\text{pos}} + \lambda_{\text{cav}} \mathcal{L}_{\text{cav}} + \lambda_E \mathcal{L}_E, \quad (15)$$

where $\mathcal{L}_{\text{pos}}$ aligns corrected positions to observation, $\mathcal{L}_{\text{cav}}$ matches the alive-scalar field via the soft-removal sigmoid, and $\mathcal{L}_E$ is the constraint energy. The cavity term contains the sigmoid in the alive-scalar expression, which structurally exposes a potential "fake gradient" path: the inner sigmoid uses (s, τ), and if those parameters were shared autograd nodes with

the outer $\boldsymbol{\tau}_{pp}$ field, the outer optimization could see a self-consistency signal that does not correspond to true physical sys-ID (*cf.* Liang's k'_c separation argument, §**??**).

We sever this path with a `.item()` detach pattern: the inner loop reads s and τ as scalar Python floats by calling `.item()` on the outer tensors, preserving the numerical value (so the inner residual mapping is consistent with the outer obs generator) but breaking the autograd link. The outer $\boldsymbol{\tau}_{pp}$ field consequently receives gradient only through the forward simulation, not through the inner formula. This detach pattern is universally safe regardless of which subset of θ is in the outer optimizer, replacing the alternative (using fixed inner constants s', τ' disjoint from outer values, which introduces a model mismatch at the inner-outer boundary).

e) Mode C state injection across frames.

Following Liang [**?**] Algorithm 1 line 16, the per-frame residuals are injected into the simulation state before the next forward call:

$$\begin{aligned} \mathbf{p}_i &\leftarrow \mathbf{p}_i + (\Delta^*_{\mathbf{x}})_i, \\ d_i &\leftarrow \max(d_i + (\Delta^*_{\mathbf{d}})_i,\, 0), \\ \alpha_i &\leftarrow \sigma\big(-s \cdot (d_i - \tau_{pp,i})\big). \end{aligned} \tag{16}$$

This online state update bakes each residual into the evolving trajectory, distinguishing our calibration regime from offline trajectory optimization that would discard the corrections. The injected α uses the current per-particle $\boldsymbol{\tau}_{pp}$ (detached to a numpy array), maintaining consistency with the forward solver's own alive-scalar computation.

f) Validation paradigm.

Because the calibrated $\mathbf{k}_{pp}$ is a PBD relaxation rate rather than a material modulus (§**??**), we validate via three complementary, paradigm-appropriate criteria rather than numerical material-constant comparison:

(a) *Spatial rank ordering.* For phantoms with a known-location embedded inclusion (3D-printed reference), we verify that the recovered field is correctly ordered: $\overline{k}^{\text{incl}}_{pp} > \overline{k}^{\text{bg}}_{pp}$, consistent with the inclusion being mechanically stiffer than the background phantom material.

(b) *Spatial IoU.* The top-$|S|$ particles by recovered k_{pp} overlap with the geometric ground-truth inclusion mask S: $\text{IoU}(\text{top}_{|S|}(\mathbf{k}_{pp}), S) > \tau_{\text{IoU}}$.

(c) *Cross-trajectory generalization.* Calibrating on one knife trajectory and predicting deformation on a held-out trajectory; sim prediction accuracy on the held-out trajectory is the calibration's test of having captured spatial material structure rather than trajectory-specific overfit.

A numerical mapping between $\mathbf{k}_{pp}$ and a real Young's modulus in physical units would require a separate forward identifiability study across calibrated specimens; we deem this out of scope and discuss it as future work in §**??**.

g) Friction and damping.

Friction μ and damping β fall outside the calibration scope, because their identifiability requires nonzero particle velocity that neither a quasi-static loading nor a single-pass cut provides reliably. We adopt literature values from Misra *et al.* [**?**] and Mackle *et al.* [**?**], following the same protocol as DiSECt [**?**] and Kolawole's cardiac inverse FEM [**?**]. Joint calibration of μ, β together with $(\mathbf{k}_{pp}, \boldsymbol{\tau}_{pp})$ from time-series force signals is reserved for future work once F/T sensor data is available (see §**??**).

The remaining ingredients of the calibration pipeline—the α-coupling principle that eliminates transition-zone artifacts and the design trade-offs against constraint-based residual mapping frameworks—are detailed in §**??** and §**??** respectively. Empirical recovery behavior of $\mathbf{k}_{pp}$ and $\boldsymbol{\tau}_{pp}$ in synthetic inclusion experiments, and the corresponding spatial-rank IoU metrics, are reported in §**??**.